

区块链基础 - Lecture 1

Foundations of Blockchains课程 (by Prof. Tim Roughgarden) 笔记

*本课程适合有计算机科学基础的读者，读者需具备编程能力和基本的数学能力，基础的数据结构与算法、操作系统原理、计算机网络以及具备基本的分布式系统知识。

*本课程不涉及具体的工程代码，而是专注于区块链技术本身的底层原理和系统分析。

*如果读者想要习得区块链协议的设计能力，那么笔者建议本课程中的证明部分不要跳过，读者证明的部分也需要多加练习。

*本文中的部分英文术语不采用中文文献中的既有翻译，笔者认为有一些现有的中文翻译有待修正。

*本笔记定有遗漏错误之处，请务必观看视频教程和教授编写的讲义（下述每个章节的开头都有讲义链接），并阅读相关文献（一定不能跳过文献阅读）

课程视频资源 (https://youtube.com/playlist?list=PLEGCF-WLh2RLOHv_xUGLqRts_9JxrckiA&si=OZF0s4r9lsS4ejXz)

0 术语表与阅读材料

0.1 术语和缩略表

f : fault 错误的，拜占庭的；

fn : fault node 拜占庭节点；

s : sender 发送者；

sn : sender node 发送者节点；

ns : non-sender 非发送者节点；

fns : fault non-sender 拜占庭的非发送节点；

h: honest 诚实的；

hn: honest node 诚实节点；

hs: honest sender 诚实的发送者；

hns: honest non-sender 诚实的非发送者；

0.2 阅读材料

James R. Lee. (2024). *Chapter 3: Byzantine Broadcast and the Dolev-Strong Protocol*. Course notes, CSE 422, University of Washington. https://homes.cs.washington.edu/~jrl/cse422wi24/notes/blockchain_3.pdf

M. Pease, R. Shostak, L. Lamport. (1983). *Reaching Agreement in the Presence of Faults*. Journal of the ACM (JACM), 27(2), 228-234.

M. J. Fischer, N. A. Lynch, M. S. Paterson. (1985). *Impossibility of distributed consensus with one faulty process*. Journal of the ACM (JACM), 32(2), 374-382.

A. Miller, Y. Xia, K. Croman, E. Shi, and D. Song.(2016). *The honey badger of BFT protocols*. Proceedings of the ACM SIGSAC Conference on Computer and Communications Security, 31-42.

C. Dwork, N. Lynch, L. Stockmeyer. (1988). *Consensus in the presence of partial synchrony*. Journal of the ACM (JACM), 35(2), 288-323.

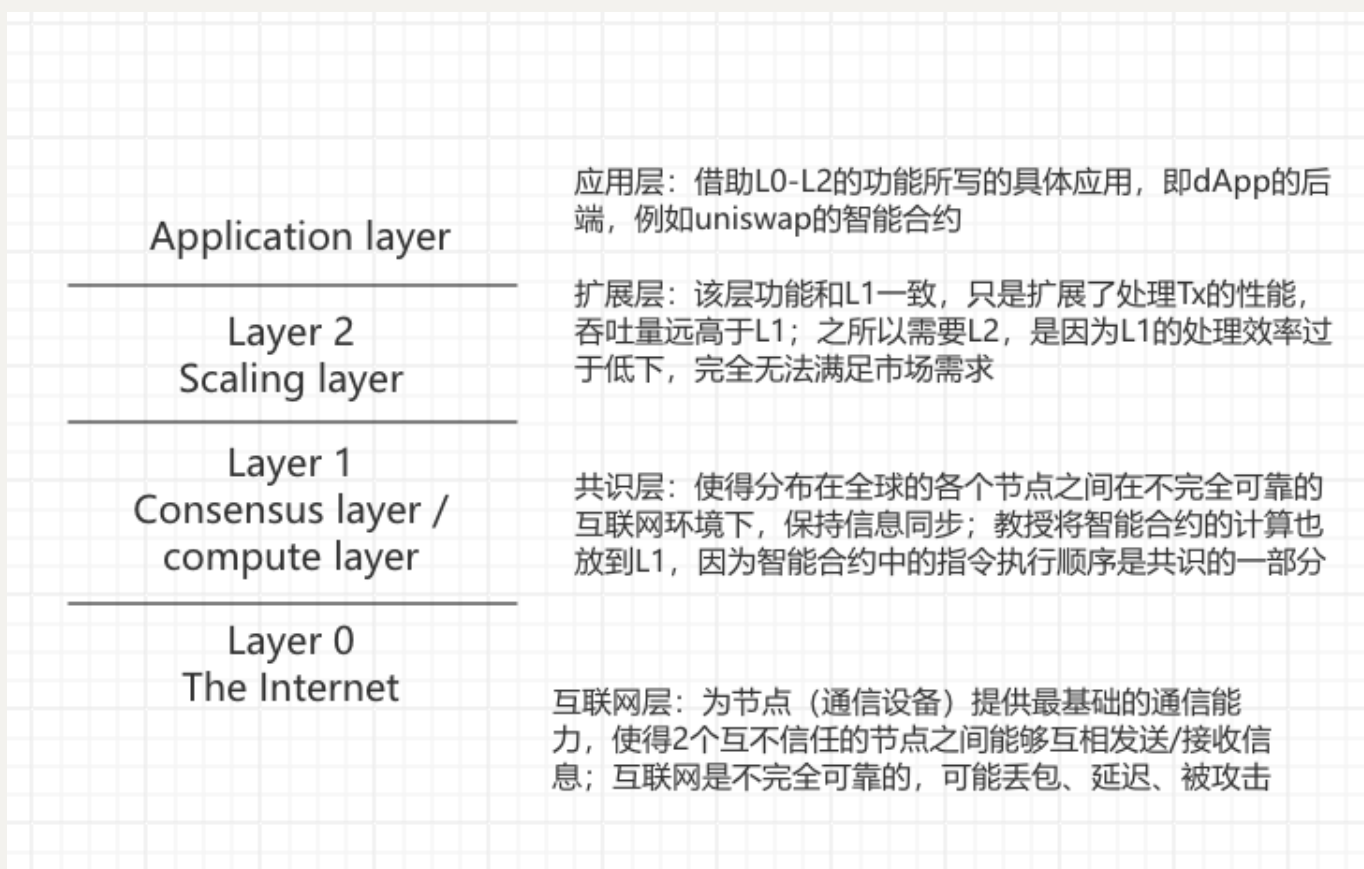
1 引论

Lecture 1 原版讲义链接: <https://timroughgarden.github.io/fob21/1/11.pdf>

1.1 课程概要

1.1.1 区块链栈 - Blockchain stack

以下stack并不是行业中所讨论的标准化的stack，而是Prof.自己总结的为了方便讲课和理解区块链原理使用的stack。



笔者补充：

L1层是区块链系统最重要的一层。我们都知道区块链是一个分布式账本（distributed ledger），之所以称为账本，是因为作为区块链的第一个成功的应用系统——比特币系统，其区块链中记录的信息是UTXO交易信息，也就是账本。后来，我们将区块链的概念进行了延伸和拓展，从结构上看区块链，实际上它是一个使用哈希值作为reference的链表，是一种数据结构。

也就是说，这个数据结构中具体记录什么样的数据其实都是可以的，并不一定非要记账，也可以是其他的数据。*因此，区块链不再只是账本！* 这个数据结构有一个特点，具有不可篡改性，因为数据在被编码为区块的时候，添加了数据本身的哈希信息。所以我们将过去的这些数据的历史，称为history，即数据历史，也就是旧版本的数据。区块链系统作为一个分布式系统，为了查询和写入当前版本的数据，并且为了保证数据的确定性、可验证性，我们可以使用状态机（State Machine）来达成这些目的。

作为一个分布式系统，区块链要解决系统可靠性问题，很容易想到使用副本的架构。而状态机很容易应用到副本架构上，也就是State Machine Replication。

故，区块链系统可以等价于状态机+不可篡改的数据历史（或者说**append-only log**）+共识机制

区块链元素	状态机中的角色
全网的一致账本（链）	状态转移历史的全记录（日志）
区块中的交易	输入（ <code>commands / inputs</code> ）
区块链节点的状态	状态机的状态
执行交易生成的新状态	状态转移函数的输出（ <code>new_state</code> ）
共识算法（如PoW、PoS）	保证状态机在多个副本间的一致性

SMR带来的问题是，我们如何确保整个系统最终所有节点在有限的时间内能够输出同一个确定的值？如何防止不同的节点输出的值是不同的？由于区块链系统是一个公开的系统，所有人都可以参与进来，难免存在一些恶意节点故意对系统进行攻击，那么如何防止其他的好节点被欺骗从而输出一个与其他节点不同的值，从而造成系统崩溃？这些都是SMR问题，也就是本课程着重需要讲解和讨论的问题。这些问题可以归结为2个基本问题：如何保证SMR系统的一致性（consistency）和活性（liveness）。解决这些问题的机制，我们称为共识机制，将这个机制写成每个节点都运行的分布式程序称为共识协议。共识协议往往是事件驱动的代码。

1.2 课程大纲 - Outline

Layer 0会被忽略，该课程假设读者学习过计算机网络课程，L0是计网的内容。

- L1的内容占本课程的60%，包括：
 - 经典共识协议 - 1980s的共识协议，即**permissioned consensus protocols**（有许可共识协议，即需要ACL的共识协议）

- 中本聪的最长链共识协议，即longest-chain consensus，也就是比特币所使用的共识协议
- 如何转向为permissionless consensus protocol（免许可共识协议，即无需ACL的共识协议）
 - 即如何抵抗女巫攻击（sybil-resistance），策略包括 Proof of Work & Proof of Stake
 - 网络难度调整，自私挖矿
 - 设计免许可共识协议的基本原则
 - 深入分析比特币的协议和以太坊的协议
- L2的内容占本课程的20%，包括：
 - 如何通过payments channels扩展比特币网络，即闪电网络（lightning network）
 - 如何通过roll-up策略扩展以太坊
 - 下一代的L1协议，其性能已经足够高，可以无需L2
- 应用层的内容占本课程的20%，包括：
 - DeFi原语（primitives），包括预言机 即 oracles，稳定币 即 stablecoins 等设计
 - 基于DeFi原语之上构建的DeFi应用，包括借贷、自动做市商（AMM）的交易系统
 - MEV 即 Maximal Extractable Value，最大可提取价值，指的是通过在区块中添加和排除交易并更改区块中的交易顺序，可以从区块生产中提取的超过标准区块奖励和燃料费用的最大值来获得收益
 - DAO 即 Decentralized Autonomous Organization，去中心化自治组织，是链上治理的一种组织，通过去中心化方式对链的变更进行投票并决策

Prof.对本课程的注解

1. 在本课程中，**区块链无关于支付和虚拟货币**，本课程专注于将区块链作为一种全新的计算范式（computing paradigm）来看待。想象一个云上计算机/计算资源，不属于任何一个特定的人或组织，而是属于全球成千上万的民众，需要为计算资源付费但无需任何额外的许可；

2. 在本课程中，区块链无关于数字货币，本课程将专注于把区块链作为实现 **permissionless programmable computer** 作为首要目标，数字货币只是实现这个目标的一种手段或副作用；之所以需要数字货币，是为了解决实现这个目标的过程中可能出现的2个问题：
 - a. 计算需求超过计算机可提供的容量时，需要使用付费的手段进行限制以防滥用，而数字货币作为这个计算机的原生货币是最为简便的支付手段
 - b. 计算机的运行需要对节点进行激励才可持续，同样的，数字货币作为这个计算机的原生货币是实现 **permissionless** 最为简便的激励方式
3. 本课程专注于讨论协议的设计原则，而不是具体协议

1.3 数字签名方案 - Digital Signature Scheme

为什么需要共识 (**consensus**) ?

非正式的解释是，共识是为了维持很多机器（即节点）对 **history** 保持同步，即使面对不可靠的网络和恶意攻击，**keep multiple machines(nodes) in sync, even in the face of an unreliable network, malicious attacks.**

在构建共识协议之前，需要一些持久假设 (**permanent assumption**) :

- 互联网一直存在，即，存在一个半可靠的机制使得不可信节点之间的点对点通讯是可能的；
- 密码学一直存在，即，存在数字签名方案和密码学哈希算法可供节点使用。

什么是数字签名方案 (**DSS**) ?

数字签名方案由三种高效的算法构成：

- 密钥生成算法 (**key generation algorithm**)，将一个随机种子 (**random seed**) 映射为 公钥-私钥 密钥对 (**(pk, sk) pair**)
- 签名算法 (**signature algorithm**)，将 消息+私钥 映射为 消息+签名，以供验证算法进行验证
- 签名验证算法 (**verification algorithm**)，将 消息+签名+公钥 映射为 "yes"/"no" (或者 "accept"/"reject")，以表示给定消息所属签名的真伪

注：从数字签名算法可知，不同的消息会产生不同的签名，这点和物理世界中用笔签名很不一样

本系列课程对于签名的一个假设：

签名是理想的，即攻击者几乎没有可能获取私钥，也就不可能对某个消息生成伪造签名

在现实中，只要密钥的长度足够长，那么这个假设几乎是正确的。但理论上，这个假设是错误的，至少使用暴力破解/枚举的手段在理论上是可以不断尝试并获取到私钥的。密码学能做的是让这种破解手段需要付出的代价和成本远高于破解后的收益，从博弈论和经济学角度出发，这样做是不符合经济人假设的，即不划算的，也就让信息保护得以可能。例如，密钥长度足够的情况下，使得使用枚举方法在现如今和可预见的未来几十年中人类所拥有的计算能力需要上万年甚至上亿年才能枚举完成，这就成了几乎不可能的破解任务。同时，需要确保DSS的三种算法是高效的，这样才有实用价值，并且从另一个维度提升安全性——人们可以不断的更替密钥，并且在被破解之前就能高效实用验证算法进行验证，否则该方案就是不安全的。另一种更为聪明的方法是通过逆向工程的方法进行破解，效率更高，将指数级别的复杂度降低到线性，能够通过这种方式被破解的密码学算法同样是不安全的。

区块链常用的密码学算法是离散对数算法，该算法/难题可以假设无法找到一个有效的逆向工程算法通过消息+签名来得到密钥。关于离散对数算法的细节，请参考《密码学原理与实践（第三版）》，Douglas R. Stinson著。

1.4 状态机复制问题 - The SMR Problem

为了在后续课程中讨论共识协议的正确性，需要介绍一些概念和问题。这里先介绍SMR - State Machine Replication，即状态机复制。

状态机是CS中已经学习过的概念，这里不再赘述。Replication（复制/副本）的概念，在DevOps和软件工程实践中也是基础概念，有过工作经验的软件工程师应当熟知，因此这里也不再赘述（例如数据库副本集，多实例数据库冗余机制等）。

我们知道，副本集/多实例冗余的问题在于，如何保持副本之间的同步，并且确定某一个共同的状态，亦即共识？

区块链系统是一个分布式系统，同样需要replication以保证系统的可靠性。区块链系统和数据库系统不一样的地方在于，数据库系统通常是一家公司所掌控的，公司对数据库拥有绝对权力，当然也包括副本，因此内部作恶的情况和概率相比于区块链这样的完全开放式的系统要小得多。区块链系统是public的，其replication需要面对public的攻击，如有利可图即存

在作恶节点的可能性。那么如何保证区块链系统的大多数节点的状态是一致、真实、未篡改并且可靠的？这就是SMR问题。

我们对SMR问题的描述再详细一点：

区块链系统中，可以大致范围两类角色——

- 一类是全节点，即服务提供者，提供计算、数据存储服务，全节点之间会使用正确的顺序（order）同步最新的仅追加数据（append-only data，亦可称其为history），或者叫做状态（state，亦即数据），或者叫做transaction（可翻译成交易或事务，缩写为tx）
- 另一类是客户端节点，它们希望从全节点上获取自己想要的数据或者写入数据，亦即提交tx。

例如，有全节点A、B和C。Alice在A节点上提交了一个tx，转账5个数字货币给Bob，然后再转账1个数字货币给Tim。那么，Alice的本地节点会产生2个tx，顺序是：prefix tx-(N) => tx-(N+1)(5 to Bob) => tx-(N+2)(1 to Tim)。这个tx会提交给A节点，A节点上的区块顺序也必须是prefix tx-(N) => tx-(N+1)(5 to Bob) => tx-(N+2)(1 to Tim)，A节点将新的两个tx同步给B和C，B和C上的区块链也是按照同样的顺序将区块添加入链中。如何做到这一点，便是SMR问题。

一般的，我们使用一种叫做协议（protocol，更准确的说，是分布式协议，即distributed protocol）的解决方案来达成这个目的。

协议，即在每个节点上都会运行的基于事件驱动的代码，这段代码会在本地进行运算，可以从别的节点接收新的消息，发送新的消息给别的节点，根据消息做出一些动作，例如改变状态，亦即往区块链中添加新的tx。

这样的协议的需要达成下述2个目标，可称其解决了SMR问题：

1. 使系统具备安全性（**safety properties**），即坏事不会发生。在区块链系统中，这样的安全属性即一致性（consistency），指的是所有节点都保持history的同步（包括tx数据和顺序的一致），允许少部分节点有所延迟，但延迟节点的history必须是其他节点的前驱（prefix），也就是不能出现数据篡改；
2. 使系统具备活性（**liveness properties**），即好事一定发生。在区块链系统中，这样的活性属性指的是节点在限定时间内必须完成对合法tx在链上的添加动作，业界称为出块。若非如此，这个系统就失去了作用，tx没有被入链，那么对用户来说系统就不可用了。